

Technische Evidenz und Claim-Grenzen

Source-role map, boundaries and reviewer proof

Sanitisierte Quellenkarte

Diese Appendix ordnet die Evidenz nach Rolle und Grenze, ohne private Dateipfade oder Zugangsdaten offenzulegen.

Layer	Evidence contribution	Boundary
Remote WhatsApp-to-Agent Command Loop	Workflow-Notizen zu WhatsApp-Event-Erkennung, Audio-Transkription, strukturierten Arbeitsaufträgen, Modellhinweisen und Quality-Loop-Schwellen.; Pipeline-Notizen zur Kette WhatsApp -> Gateway -> Watcher -> MiguelAgent -> Ausführung/Bewertung/Formatierung -> Antwort.	Persoenlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehaerteten Sicherheitsprodukts.
Miguel Core Unified Daemon	Miguel-Core-Daemon als Konsolidierung früherer Daemon- und Command-Flächen in eine lokale Kontrollschicht.; Route Map für Chat, Streaming, Voice, State, Vault, Knowledge, Prediction, Cascade, Agents, Orchestra, macOS/Tool-Control, Conversations und Health.	Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.
Multi-Model Cascade Router	AI-Router mit Provider-Registry für lokale Modelle, freie APIs, bezahlte/CLI-Modelle und LiteLLM-gestützte Calls.; Cascade-Logik mit lokalen/freien/bezahlten Ebenen, Quoten, Cooldowns, Quality Gates, Free-Model-Racing, Cancellation und Budgetgrenzen.	Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.
Memory, Vault, CASS und Smart Memory MCP	Memory Broker mit Broadcast-Schema, Write Logs, Weight/Centrality Scoring und leichter Widerspruchserkennung.; CASS-Pipeline mit episodischem Rohgedächtnis, Working Summaries, prozeduralen Regeln, Confidence, Decay und SQLite-Persistenz.	Praktische Memory-Infrastruktur, keine Behauptung menschenähnlichen Bewusstseins.
Voice Interface and Spoken Tool Use	Voice-Source mit Mikrofonstreaming, Gemini-Live-Konfiguration, Tool Declarations, Dispatcher, Vault Tools, Deep-Think-Route, begrenzter Shell und Fallback.; Voice-Pipeline-Notizen zu Wake Word, lokaler STT, hybrider TTS, Barge-in, Latenzbudget und Privacy-Grenzen.	Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.

Layer	Evidence contribution	Boundary
ANIMA Experimental Kernel	ANIMA-Python-Package mit Kernel, State, Primitives, Temporal Modules, Metrics, Bridge/Shell-Modulen, Tests und Benchmarks.; Evidence-Status- und Methodology-Dateien, die Claims begrenzen und aktuelle Benchmark-Limits benennen.	Kein Beweis fuer kuenstliches Bewusstsein; ein ambitioniertes, getestetes Experimentierfeld.

Wiederverwendbares Architekturpattern

Eingang wird von Interpretation, Ausführung, Modellwahl, Memory, Bewertung, Audit und Antwort getrennt.

- Remote-Nachrichten und Voice-Befehle sind Authority-Surfaces.
- Modellrouting wird als Governance behandelt.
- Memory wird als Provenance- und Reliability-Infrastruktur behandelt.

Technische Lesekarte: Remote WhatsApp-to-Agent Command Loop

Was ich mir dabei gedacht habe, war einfach, aber technisch anspruchsvoll: Ich wollte nicht am Laptop sitzen müssen und trotzdem sinnvoll Arbeit an ihn übergeben können, ohne mein Handy zu einer blinden Remote-Shell zu machen. Der interessante Teil war die Vertrauensgrenze. Eine Nachricht musste erst zum Arbeitsauftrag werden, ausgeführt und danach bewertet werden, bevor sie zu mir zurückkommt.

- WhatsApp oder Audio kommt in der OpenClaw-Schicht an.
- Ein Watcher normalisiert Ereignisse, dedupliziert Nachrichten und behandelt Medien.
- MiguelAgent macht daraus Ziel, Komplexitaet, Arbeitsauftrag, Erfolgskriterien und Modell-Hinweis.
- Die Ausfuehrungsseite nutzt Miguel/EXOVIUM-Tools und Modellrouting.
- Eine Qualitaetsschleife prueft, ob das Ergebnis wirklich fertig ist.
- Erst danach wird die Antwort wieder an WhatsApp geschickt.
- Workflow-Notizen zu WhatsApp-Event-Erkennung, Audio-Transkription, strukturierten Arbeitsaufträgen, Modellhinweisen und Quality-Loop-Schwellen.
- Pipeline-Notizen zur Kette WhatsApp -> Gateway -> Watcher -> MiguelAgent -> Ausführung/Bewertung/Formatierung -> Antwort.
- Python-Watcher- und MiguelAgent-Module mit Event-Parsing, Medienhandling, Deduplication, Arbeitsauftragsausführung, Evaluator-Loop und Antwortformatierung.
- Authority Boundary Thinking: Der Remote-Kanal ist bequem, aber das System behandelt ihn als riskante Kontrollfläche.
- Operatives Denken: schnelle Bestätigung, Deduplication, Medienhandling und finale Rückmeldung sind getrennte Fehlerpunkte.
- Evaluator-Loop-Intuition: Das System führt nicht nur aus, sondern prüft, ob das Ergebnis den ursprünglichen Auftrag erfüllt.

Technische Lesekarte: Miguel Core Unified Daemon

Miguel Core entstand aus einer sehr praktischen Frustration: Ich hatte zu viele starke Teile als einzelne Skripte und Services. Ich wollte daraus eine lokale Kontrollschicht machen, die beobachtbar, routbar und verständlicher ist.

- Ein Python-Daemon buendelt Chat, Streaming, Voice, State, Vault, Knowledge und Tools.
- Routen verbinden Memory Broker, CASS, Vault Watcher, Prediction, Metacognition und macOS Bridge.
- Orchestra delegiert Aufgaben an Worker und verfolgt Status, Timeout, Ergebnis und Abbruch.
- Start- und Handover-Dateien dokumentieren Modellrouting, Boot Checks und Autonomie-Ringe.
- Miguel-Core-Daemon als Konsolidierung früherer Daemon- und Command-Flächen in eine lokale Kontrollschicht.
- Route Map für Chat, Streaming, Voice, State, Vault, Knowledge, Prediction, Cascade, Agents, Orchestra, macOS/Tool-Control, Conversations und Health.
- Orchestra-Worker-Modul mit asynchroner Delegation, Timeouts, Cancellation, Status und Result Capture.
- Architekturkonsolidierung: Fragmentierung erkennen und viele Flächen in eine kontrollierbare Schicht bringen.
- Operative Reife: Boot Probes und Handover-Dokumente zeigen, dass das System betrieben werden sollte, nicht nur als Idee existiert.
- Boundary-Reife: Autonomy Rings trennen, was das System darf, von dem, was Nutzerfreigabe braucht.

Technische Lesekarte: Multi-Model Cascade Router

Ich wollte Modellwahl nicht als Geschmack behandeln. Routing sollte Teil der Architektur sein: günstig, wenn möglich; stark, wenn nötig; lokal, wenn Privacy zählt; fallback-fähig, wenn ein Provider ausfällt.

- Task-Strategien waehlen schnell, hochwertig, gratis, lokal, Code, kreativ oder bulk.
- Lokale Modelle und kostenlose APIs werden dort bevorzugt, wo es sinnvoll ist.
- Quoten, Cooldowns und Ausfaelle blockieren schlechte Routen.
- Qualitaetsgates entscheiden, ob eskaliert wird.
- Mehrere freie Modelle koennen gegeneinander laufen, langsamere Calls werden abgebrochen.
- AI-Router mit Provider-Registry für lokale Modelle, freie APIs, bezahlte/CLI-Modelle und LiteLLM-gestützte Calls.
- Cascade-Logik mit lokalen/freien/bezahlten Ebenen, Quoten, Cooldowns, Quality Gates, Free-Model-Racing, Cancellation und Budgetgrenzen.
- LiteLLM-Konfiguration mit getrennten lokalen, freien API- und bezahlten Ebenen.
- Provider-neutrales Denken: Kein einzelnes Modell wird als magisch behandelt.
- Governance-Denken: Kosten, Qualität, Privacy und Quoten gehören zur Modellwahl.
- Messpfad: Outcome Logging schafft die Grundlage für spätere empirische Routing-Entscheidungen.

Technische Lesekarte: Memory, Vault, CASS und Smart Memory MCP

Die Memory-Arbeit entstand aus der Beobachtung, dass Agenten nicht nur mehr Kontext brauchen. Sie brauchen Memory-Hygiene: Rohfakten, Zusammenfassungen, prozedurales Lernen, Widersprüche, Veraltetheit und Provenance.

- CASS trennt episodische Rohdaten, Arbeitsgedaechtnis und prozedurale Regeln.

- Memory Broker verteilt wichtige Fakten und markiert leichte Widersprüche.
- Vault Watcher denkt Dateiaenderungen, inkrementelles Indexing, Graph Updates und Staleness.
- Smart Memory MCP macht Lernen, Recall, Stats, Muster und Vorschläge als lokale Tools verfügbar.
- Memory Broker mit Broadcast-Schema, Write Logs, Weight/Centrality Scoring und leichter Widerspruchserkennung.
- CASS-Pipeline mit episodischem Rohgedächtnis, Working Summaries, prozeduralen Regeln, Confidence, Decay und SQLite-Persistenz.
- Vault-Watcher-Design mit File-Change-Detection, inkrementeller Indexierung, Graph Updates, Staleness Propagation und Forgetting Score.
- Kontextintegrität: Zusammenfassungen sind nützlich, aber gefährlich, wenn sie Rohbelege ersetzen.
- Widerspruchsbehandlung: Konflikte werden nicht einfach überschrieben.
- Agent-Safety-Relevanz: Stale Memory kann zu falschen Aktionen, falscher Sicherheit und Zielabweichung führen.

Technische Lesekarte: Voice Interface and Spoken Tool Use

Voice war spannend, weil sich das System dadurch direkt anfühlte. Genau deshalb ist es aber riskant. Wenn ich spreche und der Computer handelt, dann sind Mute, Bestätigung, Shell-Grenzen und Privacy kein Zubehör, sondern Teil des Produkts.

- Python streamt Mikrofon-Audio in Gemini Live.
- Tools erlauben Vault Search/Read/Write, Deep Thinking, Brain Status und begrenzte Shell.
- Gefährliche Shell-Muster werden blockiert, Ausführung bekommt Timeouts.
- Spätere Planung umfasst Wake Word, lokale STT, Hybrid-TTS, Barge-in und Privacy-Grenzen.
- Voice-Source mit Mikrofonstreaming, Gemini-Live-Konfiguration, Tool Declarations, Dispatcher, Vault Tools, Deep-Think-Route, begrenzter Shell und Fallback.
- Voice-Pipeline-Notizen zu Wake Word, lokaler STT, hybrider TTS, Barge-in, Latenzbudget und Privacy-Grenzen.
- VoiceBridge-Notizen zur Trennung von Browser/Orb-STT und lokaler Daemon-Antwortlogik.
- Realismus im Human Interface: Voice senkt Reibung so stark, dass Authority Checks stärker werden müssen.
- Privacy-Realismus: Always-listening-Systeme sollten lokal und mit klaren Grenzen arbeiten.
- Safety-Realismus: Barge-in ist Kontrollfunktion, kein Luxus.

Technische Lesekarte: ANIMA Experimental Kernel

ANIMA war mein Versuch, eine sehr ambitionierte Idee in Code, Tests und Evidenz zu zwingen. Der Punkt ist nicht, Bewusstsein zu behaupten. Der Punkt ist zu zeigen, dass selbst eine unscharfe Idee in Architektur, Metriken und Grenzen übersetzt werden kann.

- Kernel-, State- und Typmodule bilden den Kern.
- Primitiven wie Qualia, Engram, Valence, Nexus, Impulse, Trace, Mirror und Flux werden testbar gemacht.
- Temporal- und Metrikschichten erlauben Messung statt reiner Philosophie.
- Benchmark- und Evidence-Status-Dateien verhindern ueberzogene Claims.
- ANIMA-Python-Package mit Kernel, State, Primitives, Temporal Modules, Metrics, Bridge/Shell-Modulen, Tests und Benchmarks.
- Evidence-Status- und Methodology-Dateien, die Claims begrenzen und aktuelle Benchmark-Limits benennen.

- Lokaler Rebuild-Testlauf mit 446 bestandenen Tests und einer Konfigurationswarnung.
- Ambition mit Zurückhaltung: hoch greifen, ohne künstliches Bewusstsein als bewiesen zu verkaufen.
- Messinstinkt: Benchmarks und Ablationen existieren auch dann, wenn Resultate noch nicht entscheidend sind.
- Evidenzhierarchie: Resultate und Methodik stehen über optimistischer Überblickssprache.